

DATABASE MANAGEMENT SYSTEMS

MODULES 1–3

Exam-focused notes, SQL examples, ER diagrams, normalization, comparisons, and question bank

Exam Focus

How to use these notes. These notes consolidate the supplied lecture slides into three exam-oriented modules: **Module 1: Database Concepts + E–R Modelling**, **Module 2: SQL + SQL Functions**, and **Module 3: Functional Dependencies + Normalization**. For long answers, write the definition first, then the classification/conditions, then the mechanism or diagram, followed by a source-based example.

Contents

1	Module 1 – Database Concepts and E–R Modelling	2
1.1	1.1 Data, Database and DBMS	2
1.2	1.2 Database Applications	2
1.3	1.3 Hierarchy of Data	2
1.4	1.4 Entities, Attributes and Keys	2
1.5	1.5 Traditional File-System Approach	3
1.5.1	Drawbacks of the file-system approach	3
1.6	1.6 Database Approach	3
1.7	1.7 Why Use a DBMS?	3
1.8	1.8 Levels of Data Abstraction	3
1.9	1.9 Schema and Instance	3
1.10	1.10 Database Users	4
1.11	1.11 Database Administrator	4
1.12	1.12 Data Models	4
1.13	1.13 Database Languages	4
1.14	1.14 Relational Model	4
2	Module 1B – E–R Modelling	5
2.1	1.15 Building Blocks of an ERD	5
2.2	1.16 Chen and Crow’s Foot Notation	5
2.3	1.17 Entity	5
2.4	1.18 Relationship	5
2.5	1.19 Cardinality	5
2.6	1.20 Attribute	6
2.6.1	Types of attributes	6
2.7	1.21 ERD Symbols to Memorize	6
2.8	1.22 Strong and Weak Entities	6
2.9	1.23 Self-Relationships and Roles	6
2.10	1.24 Binary and Higher-Degree Relationships	7
2.11	1.25 Superkey, Candidate Key and Primary Key in ER Modelling	7
2.12	1.26 Specialization and Generalization	7
2.12.1	Constraints	7
2.13	1.27 Aggregation	7
2.14	1.28 Reduction of ER Diagrams to Tables	7
2.14.1	Strong entity set	7
2.14.2	Weak entity set	7
2.14.3	One-to-one relationship	7
2.14.4	One-to-many / many-to-one relationship	8

2.14.5	Many-to-many relationship	8
2.14.6	Composite attribute	8
2.14.7	Multivalued attribute	8
2.14.8	Class hierarchy	8
2.14.9	Aggregation	8
3	Module 1 – Important Questions	9
3.1	Definitions / 2–3 marks	9
3.2	5-mark questions	9
3.3	10-mark questions	9
4	Module 2 – SQL	9
4.1	2.1 SQL Overview	9
4.2	2.2 SQL Language Categories	10
4.3	2.3 DDL	10
4.3.1	CREATE TABLE	10
4.4	2.4 SQL Domain/Data Types	10
4.5	2.5 Keys in SQL	10
4.6	2.6 Constraints	11
4.6.1	Column-level vs table-level constraints	11
4.7	2.7 ALTER TABLE	11
4.8	2.8 DROP TABLE	12
4.9	2.9 DML – INSERT	12
4.10	2.10 INSERT ... SELECT	12
4.11	2.11 DELETE	12
4.12	2.12 UPDATE	12
4.13	2.13 DELETE vs TRUNCATE	13
4.14	2.14 Basic SQL Query Structure	13
4.15	2.15 SELECT Clause	13
4.15.1	DISTINCT and ALL	13
4.15.2	SELECT * and arithmetic expressions	13
4.16	2.16 WHERE Clause	14
4.16.1	BETWEEN	14
4.17	2.17 ORDER BY	14
4.18	2.18 Set Operations	14
4.19	2.19 GROUP BY	15
4.20	2.20 HAVING	15
4.21	2.21 LIKE and String Operations	15
4.22	2.22 NULL Values	15
5	Module 2B – SQL Functions	16
5.1	2.23 Functions	16
5.2	2.24 Types of SQL Functions	16
5.3	2.25 Character Functions	16
5.4	2.26 Numeric Functions	17
5.5	2.27 Conversion Functions	17
5.6	2.28 General Functions	18
5.7	2.29 Aggregate Functions	18
5.8	2.30 Date Functions	18
5.9	2.31 Special Date Formats with TO_CHAR	19
6	Module 2 – Important Questions	20
6.1	Definitions / 2–3 marks	20
6.2	5-mark questions	20
6.3	10-mark questions	20

7	Module 3 – Functional Dependencies and Normalization	20
7.1	3.1 Redundancy and Its Problems	20
7.2	3.2 Update, Insertion and Deletion Anomalies	21
7.3	3.3 Functional Dependency	21
7.3.1	Trivial and non-trivial FD	21
7.4	3.4 Reasoning About Functional Dependencies	21
7.5	3.5 Armstrong's Axioms	21
7.6	3.6 Attribute Closure	22
7.6.1	Method	22
7.6.2	Source example	22
7.7	3.7 Candidate Keys	22
7.8	3.8 Prime and Non-Prime Attributes	22
7.9	3.9 Normal Forms	22
7.10	3.10 First Normal Form (1NF)	23
7.11	3.11 Second Normal Form (2NF)	23
7.11.1	Partial dependency	23
7.11.2	Source example	23
7.12	3.12 Third Normal Form (3NF)	24
7.12.1	Transitive dependency	24
7.13	3.13 BCNF	24
7.13.1	Source example	24
7.14	3.14 3NF vs BCNF	24
7.15	3.15 Decomposition	25
7.16	3.16 Types of Decomposition	25
7.17	3.17 Lossless-Join Decomposition	25
7.18	3.18 Dependency Preservation	25
7.19	3.19 Fourth Normal Form (4NF)	26
7.20	3.20 Decomposition for 4NF	26
7.21	3.21 Fifth Normal Form (5NF)	26
7.22	3.22 Minimal / Canonical Cover	26
7.22.1	Steps	26
7.22.2	Source example	26
8	Module 3 – Worked Exam Frameworks	28
8.1	3.23 How to Check a Relation for Normal Form	28
8.2	3.24 Quick Normal-Form Comparison	28
8.3	3.25 Important Practice Problems from the Slides	28
9	4. 10-Mark Answer Templates	29
9.1	Question A: Explain DBMS and its advantages	[10 marks] 29
9.2	Question B: Explain ER modelling	[10 marks] 29
9.3	Question C: Explain SQL commands	[10 marks] 29
9.4	Question D: Explain SQL functions	[10 marks] 29
9.5	Question E: Explain normalization and normal forms	[10 marks] 30
10	5. High-Yield Comparison Tables	31
10.1	5.1 File System vs DBMS	31
10.2	5.2 DDL vs DML	31
10.3	5.3 WHERE vs HAVING	31
10.4	5.4 Single-Row vs Aggregate Functions	31
10.5	5.5 2NF vs 3NF vs BCNF	31
11	6. Last-Minute Revision Page	32
12	7. Rapid-Fire Question Bank	33

13 8. Exam Writing Checklist**33**

Module 1 – Database Concepts and E–R Modelling

1.1 Data, Database and DBMS

Data consists of raw facts. When organized, data may be transformed into information.

Database is a collection of data organized to meet users' needs.

Database Management System (DBMS) is a group of programs that manipulates the database and provides an interface between the database and the user or other application programs.

Remember

A compact chain to remember is:

Raw Data → Organized Data → Information

and

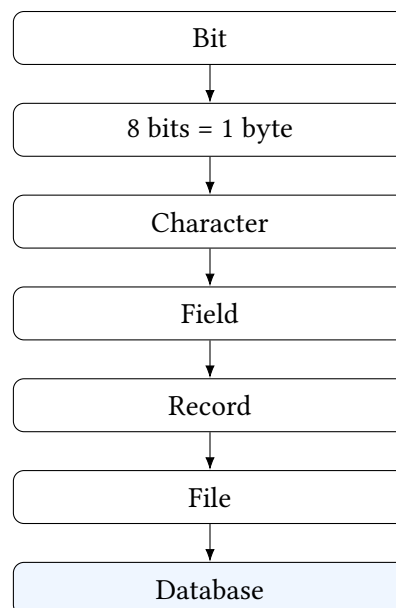
User/Application ↔ DBMS ↔ Database.

1.2 Database Applications

The introductory slides use examples such as banking, airlines, library systems, reservations, and large enterprise applications. The supplied slides also illustrate databases used by services such as shopping, banking/ATMs, railway reservation, and mobile applications.

1.3 Hierarchy of Data

The supplied material describes the data hierarchy as:



A **field** is a logical grouping of characters into a word, a small group of words, or a complete number. A **record** is a logical grouping of related fields. A **file** is a logical grouping of related records.

1.4 Entities, Attributes and Keys

Entity: a generalized class of people, places, or things for which data are collected, stored, and maintained. Examples include Customer and Employee.

Attribute: a characteristic of an entity. Examples include customer name and employee name.

Key field: an attribute used to identify records/entities uniquely.

Exam Focus

Likely 5-mark question: Explain entity, attribute, and key with suitable examples.

1.5 Traditional File-System Approach

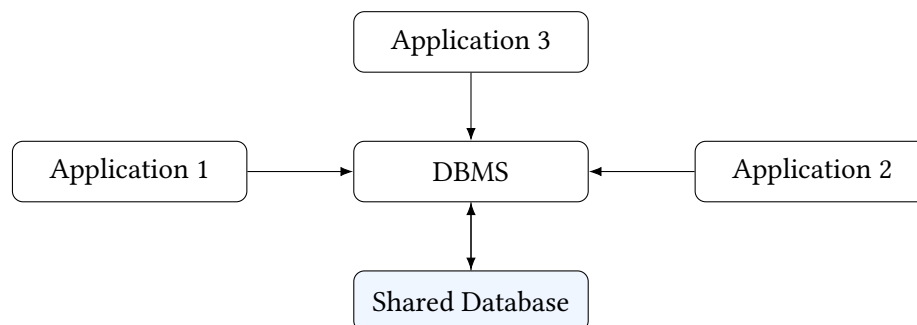
In the traditional approach, separate files are created and stored for each application program.

Drawbacks of the file-system approach

- **Data redundancy:** the same data may be stored in multiple files, wasting storage space.
- **Data inconsistency:** duplicate data may be updated in one file but not another.
- **Difficulty in accessing data:** retrieving specific information can require writing separate programs.
- **Concurrent-access problems:** simultaneous access can overwrite changes or create inconsistencies.
- **Security problems:** file systems provide limited access control compared with a DBMS.

1.6 Database Approach

The database approach uses a pool of related data shared by multiple application programs. Instead of separate application-specific data files, applications use data that are joined or related in the database.



1.7 Why Use a DBMS?

The supplied slides identify:

- data independence and efficient access,
- reduced application-development time,
- data integrity and security,
- uniform data administration,
- concurrent access,
- recovery from crashes.

1.8 Levels of Data Abstraction

1. **Physical level:** describes how a record is stored.
2. **Logical level:** describes what data are stored and the relationships among the data.
3. **View level:** describes the part of the database that a particular user/application sees.

Remember

Think **Physical** = how stored, **Logical** = what/relationships, **View** = what a user sees.

1.9 Schema and Instance

A **schema** is the logical structure of the database. The supplied slides compare this to type information in a programming language.

An **instance** is the actual data stored in the database at a particular time.

1.10 Database Users

The slides identify:

- **Database Administrators (DBAs)**
- **Application programmers**
- **Sophisticated users**
- **Naïve users**

1.11 Database Administrator

The DBA coordinates database-system activities and has a strong understanding of the enterprise's information resources and needs. The supplied material identifies duties including schema definition and storage-structure/access-method definition.

1.12 Data Models

A data model is a collection of tools for describing:

- data,
- data relationships,
- data semantics,
- data constraints.

The supplied slides mention the relational model, E–R model, object-based models, and semi-structured models such as XML.

1.13 Database Languages

Language	Purpose / examples from the slides
DDL	Defines the database schema. Examples: CREATE, ALTER, DROP.
DML	Accesses/manipulates data. Examples include INSERT, UPDATE, DELETE, SELECT.
DCL	Controls access/authorization.

1.14 Relational Model

The relational model represents data using tables.

- A **row** is a tuple / relational instance.
- A **column** is an attribute.
- A **table** is a relation/schema.
- **Cardinality** = number of rows.
- **Degree** = number of columns.

Module 1B – E–R Modelling

1.15 Building Blocks of an ERD

The supplied ERD slides give a useful grammar analogy:

ERD element	Grammar analogy	Examples
Entity	Proper noun	Student, Employee, Instructor, Course
Relationship	Verb	has, teaches, belongs, handles
Attribute	Adjective / characteristic	Age, Gender, First Name

1.16 Chen and Crow’s Foot Notation

The supplied slides show two popular notations: **Chen notation** and **Crow’s Foot notation**.



Chen: rectangle = entity, diamond = relationship

Crow’s Foot notation represents “many” using a crow’s-foot symbol and uses line markers for one, zero-or-one, one-or-many, and zero-or-many.

1.17 Entity

An entity is a real-world object distinguishable or unique from other objects. It can be concrete/physical (employee, student, faculty, customer) or conceptual/abstract (transaction, order, course, subject).

In Chen notation, an entity is normally represented by a **rectangle**.

Entities can represent:

- people,
- places,
- objects,
- events,
- ideas/concepts.

1.18 Relationship

A relationship is a way of relating one entity to another. It is commonly thought of as a verb connecting entities/nouns and is normally represented by a **diamond** in Chen notation.

Examples from the slides:

- Sales Rep **represents** Customer.
- Customer **has** Order.

1.19 Cardinality

Cardinality is the number of items that must be included in a relationship.

Minimum cardinality:

- **0** = optional role.
- **1** = mandatory role.

The supplied material covers:

- 1:1 – one-to-one,
- 1:M – one-to-many,
- M:1 – many-to-one,
- M:N – many-to-many.

Exam Focus

Exam diagram: Be able to draw and explain 1:1, 1:M and M:N with a sentence describing the relationship from both sides.

1.20 Attribute

An attribute is a characteristic, basic fact, or field of an entity or relationship. In Chen notation it is normally represented by an **oval**.

Example: Student may have ID Number, Last Name, First Name, Address, and Birth Date. A relationship can also have an attribute, such as Enrollment Date for a Student–Course enrollment relationship.

Types of attributes

- **Simple:** cannot be meaningfully divided further.
- **Composite:** can be divided into components, e.g. Name → First/Middle/Last.
- **Single-valued:** one value for each entity.
- **Multivalued:** multiple values, e.g. phone numbers or dependents.
- **Derived:** computed from another attribute, e.g. Age from Date of Birth.

1.21 ERD Symbols to Memorize

Symbol / notation	Meaning in the supplied slides
Rectangle	Entity set
Diamond	Relationship set
Line	Links attributes to entities and entities to relationships
Ellipse	Attribute
Double ellipse	Multivalued attribute
Dashed ellipse	Derived attribute
Underline	Key attribute
Double rectangle	Weak entity

1.22 Strong and Weak Entities

A **strong entity** has its own identifying key.

A **weak entity** cannot be uniquely identified by its own attributes alone and borrows identifying information from a strong entity. The slides use SECTION as an example associated with COURSE, where a discriminator such as s-id combines with the owner's key to form a unique identifier.

Do Not Confuse

Do not confuse a **weak entity** with an entity that merely has a foreign key. The supplied ER material specifically describes the weak entity as depending on a strong entity for identification.

1.23 Self-Relationships and Roles

An entity set can participate in a relationship with itself. Such self-relationships may use different role names on the lines connecting the relationship to the entity set.

1.24 Binary and Higher-Degree Relationships

The slides distinguish binary relationships from 3-way and k-way relationships. A relationship can therefore involve more than two entity sets.

1.25 Superkey, Candidate Key and Primary Key in ER Modelling

Term	Definition from the supplied material
Superkey	A set of one or more attributes whose values uniquely determine an entity.
Candidate key	A minimal superkey.
Primary key	A candidate key chosen by the DBA.

1.26 Specialization and Generalization

Specialization is a top-down design process in which subgroupings are designated within an entity set. These lower-level entity sets have attributes or relationships not applicable to the higher-level set.

Generalization is the opposite, a bottom-up process that combines entity sets sharing common features into a higher-level entity set.

Remember

Specialization: **one general entity** → **specialized subgroups**.
 Generalization: **several similar entities** → **one generalized supertype**.

Constraints

The slides describe:

- **Condition-defined** membership, e.g. all customers over 65 as senior citizens.
- **User-defined** membership.
- **Completeness constraint**: whether an entity in the higher-level set must belong to at least one lower-level set.
- **Total** completeness: every higher-level entity must belong to a lower-level entity set.

1.27 Aggregation

Aggregation treats a relationship as an “abstract” entity and allows relationships between relationships. The slides note that it is rarely used.

1.28 Reduction of ER Diagrams to Tables

The supplied material gives the following translation rules.

Strong entity set

Create one table for each entity set. Create a column for each attribute. The primary key becomes the table’s primary key.

Weak entity set

Create a table for the weak entity’s attributes, include the discriminator, and include the primary key of the owner/strong entity. The identifying combination forms the key.

One-to-one relationship

Either side can be selected as the “many” side for implementation; an extra attribute containing the other side’s primary key can be added accordingly.

One-to-many / many-to-one relationship

Add an extra attribute to the **many side** containing the primary key of the **one side**.

Many-to-many relationship

Create a new relation containing the keys of both participating entity sets. The primary key is the union of the two foreign keys.

Composite attribute

Create one column for each component attribute; do not create a separate column for the composite attribute itself.

Multivalued attribute

Create a new relation with:

- the primary key of the entity/relationship that owns the multivalued attribute,
- one column for the individual multivalued item.

Each cell in the multivalued column stores one value.

Class hierarchy

The supplied slides give two approaches depending on disjointness and completeness. For non-disjoint and/or non-complete hierarchies, create a table for the superclass and tables for subclasses. For a disjoint and complete hierarchy, the slides describe omitting the superclass table and placing superclass attributes into the subclass tables.

Aggregation

Create a table containing the primary keys of all involved entities, according to the supplied aggregation mapping.

Exam Focus

Very likely long-answer area: “Explain reduction of an ER diagram to relational tables.” Write the rule for strong entity, weak entity, 1:1, 1:M, M:N, composite attribute, multivalued attribute, and class hierarchy, with a small example.

Module 1 – Important Questions

Definitions / 2–3 marks

1. Define data, database, and DBMS.
2. What is data redundancy?
3. Define schema and instance.
4. What is a data model?
5. Define entity and attribute.
6. What is cardinality?
7. Define strong and weak entity.
8. What is a superkey?
9. Define specialization and generalization.
10. What is aggregation?

5-mark questions

1. Explain the drawbacks of the traditional file-system approach.
2. Explain the three levels of data abstraction.
3. Explain the different database users and the role of the DBA.
4. Explain Chen notation and Crow's Foot notation.
5. Explain the types of attributes in an ER model.
6. Explain mapping cardinalities with examples.
7. Explain specialization and generalization.

10-mark questions

1. Explain DBMS, its advantages, data abstraction levels, schema/instance, and database users.
2. Explain ER modelling with entities, relationships, attributes, cardinality, strong/weak entities, and ERD symbols.
3. Explain reduction of an ER diagram to relational tables for 1:1, 1:M, M:N, weak entities, composite attributes, multivalued attributes, and class hierarchies.

Module 2 – SQL

2.1 SQL Overview

SQL stands for **Structured Query Language** and is described in the supplied slides as a language for relational DBMSs.

The history slide states that the language originated as IBM SEQUEL as part of the System R project at IBM San Jose Research Laboratory and was renamed Structured Query Language. The slides list SQL standards including SQL-86, SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011, SQL:2016 and SQL:2023.

Source Note

The supplied SQL slide explicitly cautions that not all examples may work on every particular database system. The notes therefore preserve the Oracle/SQL examples as source material rather than silently changing them to another dialect.

2.2 SQL Language Categories

Category	Main commands in the slides	Purpose
DDL	CREATE, ALTER, DROP	Define/modify database structure
DML	INSERT, UPDATE, DELETE, SELECT	Access and manipulate table data
DCL	GRANT, REVOKE	Control authorization
TCL	COMMIT, SAVEPOINT, ROLLBACK	Transaction control

2.3 DDL

DDL deals with database structure. The supplied material says DDL covers the schema, domains of values, integrity constraints, indices, security/authorization information, and physical storage structure.

CREATE TABLE

```
CREATE TABLE tablename
(
    columnname datatype[(size)],
    columnname datatype[(size)]
);
```

Example pattern from the slides:

```
CREATE TABLE Employees (
    EmployeeID INT,
    AadhaarNO VARCHAR2(10),
    FirstName VARCHAR2(50),
    LastName VARCHAR2(50) NOT NULL,
    DeptID VARCHAR2(10),
    Age INT,
    Salary NUMBER(10,2)
);
```

2.4 SQL Domain/Data Types

The slides list:

- CHAR(n) – fixed-length character string.
- VARCHAR(n) / VARCHAR2(n) – variable-length character string.
- INT – integer.
- SMALLINT – small integer.
- NUMERIC(p, d) – fixed-point number with precision and scale.
- REAL, DOUBLE PRECISION – floating-point types.
- FLOAT(n) – floating-point type with specified precision.
- DATE – date type; the slides show DD-MON-YYYY or DD/MON/YYYY style.

2.5 Keys in SQL

Key	Definition	Source example
Superkey	One or more attributes that uniquely identify a row.	(SSN, Address)
Candidate key	Minimal superkey.	SSN, EmployeeID
Primary key	Chosen candidate key used as unique identifier.	EmployeeID
Foreign key	Attribute(s) referring to the primary key of another table.	DeptID → Department(DeptID)

2.6 Constraints

The supplied slides identify:

- NOT NULL
- PRIMARY KEY
- UNIQUE
- DEFAULT
- FOREIGN KEY
- CHECK

Column-level vs table-level constraints

Feature	Column-level	Table-level
Scope	Single column	One or more columns
Placement	Inside column definition	At end of table definition
Composite key	Not possible	Possible
Foreign key	Can be specified for one column	Commonly used for multi-column constraints

Example:

```
CREATE TABLE EmployeeProjects (
    EmployeeID INT,
    ProjectID INT,
    Role VARCHAR2(50),
    HoursWorked INT,
    PRIMARY KEY (EmployeeID, ProjectID),
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID),
    FOREIGN KEY (ProjectID) REFERENCES Projects(ProjectID),
    CHECK (Role IN ('Developer', 'Manager', 'Analyst')),
    CHECK (HoursWorked >= 0 AND HoursWorked <= 168)
);
```

2.7 ALTER TABLE

The image-only code slide supplied with the SQL material contains these examples:

```
-- Adding a column
ALTER TABLE Employees ADD DateOfBirth DATE;

-- Modifying an existing column
ALTER TABLE Employees MODIFY Salary NUMBER(12,2);
ALTER TABLE Employees MODIFY Name VARCHAR2(100) NOT NULL;

-- Dropping a column
ALTER TABLE Employees DROP COLUMN DateOfBirth;

-- Renaming a column
ALTER TABLE Employees RENAME COLUMN Address TO Location;

-- Renaming a table
ALTER TABLE Employees RENAME TO EmployeeDetails;

-- Adding constraints
ALTER TABLE Employees ADD PRIMARY KEY (EmployeeID);
ALTER TABLE Employees ADD UNIQUE (SSN);
ALTER TABLE Employees ADD FOREIGN KEY (DeptID)
    REFERENCES Departments(DeptID);
ALTER TABLE Employees ADD CHECK (Salary > 0);
```

```
-- Dropping constraints
ALTER TABLE Employees DROP PRIMARY KEY;
ALTER TABLE Employees DROP CONSTRAINT fkdepartment;
ALTER TABLE Employees DROP CONSTRAINT chksalary;

-- Altering a default
ALTER TABLE Employees MODIFY Age INT DEFAULT 25;
```

2.8 DROP TABLE

DROP TABLE removes an entire table, including its data and structure.

```
DROP TABLE tablename;
```

2.9 DML – INSERT

```
INSERT INTO tablename[(columnlist)]
VALUES (valuelist);
```

Source example:

```
INSERT INTO studios
VALUES (1, 'Giant', 'Los Angeles', 'CA');

INSERT INTO studios
(studiocity, studiostate, studioname, studioid)
VALUES
('Burbank', 'CA', 'MPM', 2);
```

If columns are omitted, values follow the table's column order. The slides also describe inserting NULL by omitting a nullable/defaulted column or explicitly specifying NULL.

2.10 INSERT ... SELECT

A SELECT query can supply values to an INSERT.

```
INSERT INTO citystate
SELECT studiocity, studiostate
FROM studios;

INSERT INTO citystate
SELECT DISTINCT studiocity, studiostate
FROM studios;
```

2.11 DELETE

```
DELETE FROM tablename
[WHERE condition];
```

Without a WHERE condition, all rows are deleted.

2.12 UPDATE

```
UPDATE tablename
SET columnname1 = value1,
    columnname2 = value2
[WHERE condition];
```

Example:

```
UPDATE studios
SET studiocity = 'New York',
    studiostate = 'NY'
WHERE studioid = 1;
```

Do Not Confuse

If the WHERE condition is omitted, the supplied slides state that all rows are updated.

2.13 DELETE vs TRUNCATE

Feature	DELETE	TRUNCATE
Scope	Can remove selected rows with WHERE	Removes all rows
Operation described in slides	Processes rows and attendant database tasks	Throws away all rows at once
Speed described in slides	Slower	Much quicker
Use in supplied notes	Preferred when integrity checks/dependencies matter	Used to remove all rows quickly

```
TRUNCATE TABLE studios;
```

2.14 Basic SQL Query Structure

The supplied slides give:

```
SELECT A1, A2, ..., An
FROM r1, r2, ..., rm
WHERE P;
```

where A_i represents an attribute, r_i a relation, and P a predicate. The result of an SQL query is a relation.

2.15 SELECT Clause

The SELECT clause lists desired attributes.

```
SELECT branchname
FROM loan;
```

SQL names are described as case-insensitive in the slides.

DISTINCT and ALL

```
SELECT DISTINCT branchname
FROM loan;
```

```
SELECT ALL branchname
FROM loan;
```

SELECT * and arithmetic expressions

```
SELECT *
FROM loan;
```

```
SELECT loannumber, branchname, amount * 100
FROM loan;
```

2.16 WHERE Clause

WHERE specifies conditions that result rows must satisfy.

```
SELECT loannumber
FROM loan
WHERE branchname = 'Perryridge'
    AND amount < 1200;
```

Logical connectives mentioned are AND, OR, and NOT.

BETWEEN

```
SELECT loannumber
FROM loan
WHERE amount BETWEEN 90000 AND 100000;
```

2.17 ORDER BY

The slides use:

```
SELECT DISTINCT customername
FROM borrower
WHERE branchname = 'Perryridge'
ORDER BY customername;
```

ASC is ascending and DESC is descending; ascending is the default in the supplied material.

2.18 Set Operations

The supplied material covers:

- UNION
- INTERSECT
- EXCEPT/MINUS
- UNION ALL, INTERSECT ALL, EXCEPT ALL/MINUS ALL

The first three eliminate duplicates according to the slide.

Examples:

```
SELECT customername FROM depositor
MINUS
SELECT customername FROM borrower;

SELECT customername FROM depositor
INTERSECT
SELECT customername FROM borrower;

SELECT customername FROM depositor
UNION
SELECT customername FROM borrower;
```

Source Note

The set-operation examples in the supplied slide are paired with questions such as customers with an account but no loan, and customers with both a loan and an account. Preserve the operator and the question together when studying.

2.19 GROUP BY

GROUP BY groups rows having the same value(s), allowing aggregate functions to be applied to each group.

```
SELECT branchname, COUNT(*)  
FROM account  
GROUP BY branchname;
```

The supplied slide notes that non-aggregate attributes in SELECT must appear in GROUP BY.

2.20 HAVING

HAVING filters groups after group formation.

```
SELECT branchname, AVG(balance)  
FROM account  
GROUP BY branchname  
HAVING AVG(balance) > 1200;
```

Remember

WHERE filters rows before grouping. **HAVING** filters groups after grouping.

2.21 LIKE and String Operations

The supplied material uses:

- % – matches any substring.
- _ – matches any single character.

Example:

```
SELECT customername  
FROM customer  
WHERE customerstreet LIKE '%Main%';
```

The slides also mention escaping a literal percent sign and list concatenation using —, case conversion, string length, and substring extraction.

2.22 NULL Values

NULL represents an unknown value or the absence of a value.

```
SELECT loannumber  
FROM loan  
WHERE amount IS NULL;
```

The supplied slides emphasize:

$$5 + \text{NULL} = \text{NULL}.$$

They also state that aggregate functions simply ignore NULL values.

Module 2B – SQL Functions

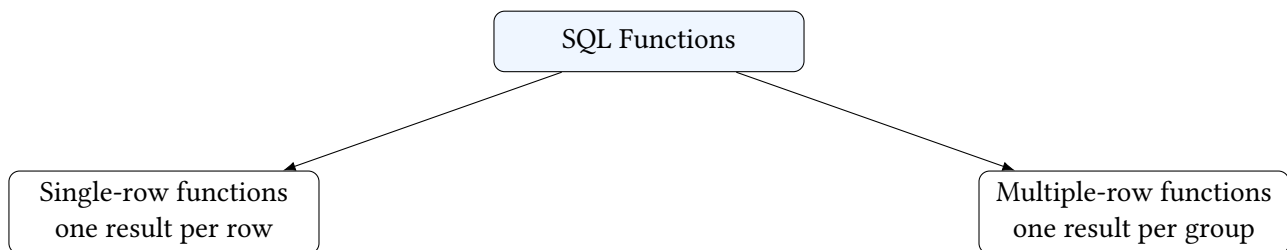
2.23 Functions

SQL functions manipulate data items and return a result. The supplied function slides describe them as similar to operators in that they manipulate data items and return a result.

Advantages listed:

- perform complex calculations,
- modify individual data items,
- manipulate output for groups of rows,
- manipulate character and numeric data,
- alter date formats for display.

2.24 Types of SQL Functions



Single-row functions can be used in SELECT, WHERE, and ORDER BY according to the supplied slides.

Syntax:

function_name[(arg1, arg2, . . .)]

Single-row categories in the slides:

- character,
- number/arithmetic,
- conversion,
- general,
- date.

Multiple-row functions are presented as **aggregate functions**.

2.25 Character Functions

Function	Purpose	Source syntax/example
LOWER	Converts letters to lowercase.	LOWER(char); LOWER(' IVAN BAYROSS') → ivan bayross
INITCAP	First letter of each word in uppercase.	INITCAP(char); INITCAP(' IVAN BAYROSS') → Ivan Bayross
UPPER	Converts letters to uppercase.	UPPER(char); UPPER(' ivan bayross') → IVAN BAYROSS
SUBSTR	Extracts a portion of a string. First position is 1.	SUBSTR(string, start, length); SUBSTR('secure', 3, 4) → cure
ASCII	Returns code for the specified character; with multiple characters, first character is used.	ASCII('a') → 97
LENGTH	Returns length of a word/string.	LENGTH('sharanam') → 8
LTRIM	Removes specified leading characters.	LTRIM('nisha', 'n') → isha

RTRIM	Removes specified trailing characters.	<code>RTRIM('sunila', 'a') → sunil</code>
TRIM	Removes specified characters from beginning/end.	<code>TRIM(' hansel ') → hansel</code>
LPAD	Left-pads a string to a specified length.	<code>LPAD('page1', 10, '*') → *****page1</code>
RPAD	Right-pads a string to a specified length.	<code>RPAD('ivan', 10, 'x') → ivanxxxxxx</code>
VSIZE	Returns bytes in internal representation.	<code>VSIZE('sct on the net') → 14 (source example)</code>
INSTR	Returns location of a substring.	<code>INSTR(string1, string2, start, nth)</code>

2.26 Numeric Functions

Function	Purpose	Source example
ABS	Absolute value.	<code>ABS(-15)</code>
POWER	m raised to the n th power.	<code>POWER(3, 2)</code>
ROUND	Rounds to specified decimal places; negative precision can round left of decimal.	<code>ROUND(15.19, 1) → 15.2</code>
SQRT	Square root.	<code>SQRT(25) → 5</code>
EXP	e^n , where source gives $e = 2.71828183$.	<code>EXP(5) → 148.413159</code>
EXTRACT	Extracts date components such as year/month/day as described.	<code>EXTRACT(YEAR FROM DATE '2004-07-02') → 2004</code>
GREATEST	Greatest value in a list.	<code>GREATEST(4, 5, 17) → 17</code>
LEAST	Least value in a list.	<code>LEAST(4, 5, 17) → 4</code>
MOD	Remainder after division.	<code>MOD(15, 7) → 1</code>
TRUNC	Truncates a number to specified decimal places.	<code>TRUNC(125.815, 1) → 125.8</code>
FLOOR	Largest integer less than or equal to a number.	<code>FLOOR(24.8) → 24</code>
CEIL	Smallest integer greater than or equal to a number.	<code>CEIL(24.8) → 25</code>

Source Note

The slides state for MOD that if the second number is zero, the result is the same as the first number. This is retained here as source wording; check the exact behavior of the target DBMS if the question asks for implementation-specific behavior.

2.27 Conversion Functions

Conversion functions convert a value from one form/datatype to another.

Function	Purpose	Source example
TO_CHAR(d,f)	Converts a date to character format according to format <i>f</i> .	SELECT SYSDATE, TO_CHAR(SYSDATE, 'DAY') FROM DUAL;
TO_DATE(char,f)	Converts a character string representing a date to a date format.	TO_DATE('06/07/2009', 'DD/MM/YY')
NVL(col,value)	Substitutes a value for NULL; replacement datatype must match column datatype.	NVL(NULL, 101) → 101

2.28 General Functions

The supplied slides cover:

- **GREATEST**: greatest expression in a list; expressions are implicitly converted according to the first expression as described in the source.
- **LEAST**: least expression in a list, with conversion behavior described similarly.
- **UID**: integer identifying the current Oracle database user.
- **USER**: returns the current Oracle user's name.

2.29 Aggregate Functions

Aggregate functions operate on multiple rows/groups.

Function	Meaning in supplied slides	Example
AVG	Average value.	SELECT AVG(sal) FROM emp;
MIN	Minimum value.	SELECT MIN(sal) FROM emp;
COUNT(expr)	Number of rows where expression is not NULL.	SELECT COUNT(acct_no) FROM acct.mstr;
COUNT(*)	Number of rows in a table, including rows with NULLs.	SELECT COUNT(*) FROM acct.mstr;
MAX	Maximum value.	SELECT MAX(curbal) FROM acct.mstr;
SUM	Sum of values.	SELECT SUM(curbal) FROM acct.mstr;

Source Note

One supplied slide contains a wording error for MAX, saying “Returns the minimum value.” The function name and example are MAX, so this note treats the phrase as a source typo rather than silently reproducing it as a definition.

2.30 Date Functions

The slides describe Oracle dates as internally representing century, year, month, day, hours, minutes, and seconds, with DD-MON-YY as the default display format in the source.

Function	Purpose	Source example
SYSDATE	Current system date/time of type DATE.	SELECT SYSDATE FROM DUAL;
ADD_MONTHS(d,n)	Adds/subtracts months from a date.	ADD_MONTHS(SYSDATE, 4)
MONTHS_BETWEEN(d1,d2)	Number of months between two dates; sign depends on ordering.	MONTHS_BETWEEN(d1, d2)
NEXT_DAY(date,day)	Date of the next specified weekday.	NEXT_DAY(SYSDATE, 'FRIDAY')
LAST_DAY(d)	Last day of the specified month.	LAST_DAY(SYSDATE)

ROUND(d,format)	Rounds a date to a specified unit.	ROUND(SYSDATE, 'MM')
TRUNC(d,format)	Truncates a date to a specified unit.	TRUNC(SYSDATE, 'YEAR')

2.31 Special Date Formats with TO_CHAR

The supplied slides specifically cover:

- DDTH – displays ordinal suffixes such as 2ND, 3RD, 8TH.
- DDSP – spells the day number, such as TWENTY-FIVE.
- DDSPTH – spells the day number and adds the ordinal form, such as TWENTY-FIFTH.

Examples:

```
SELECT custno,  
       TOCHAR(dobinc,'DDTH-MON-YY') AS DOBINC  
FROM custmaster;  
  
SELECT custno,  
       TOCHAR(dobinc,'DDSP') AS DOBDDSP  
FROM custmaster;  
  
SELECT custno,  
       TOCHAR(dobinc,'DDSPTH') AS DOBDDSPTH  
FROM custmaster;
```

Module 2 – Important Questions

Definitions / 2–3 marks

1. Define DDL, DML, DCL and TCL.
2. What is a primary key?
3. What is a foreign key?
4. Define column-level and table-level constraints.
5. What is the purpose of DISTINCT?
6. Define GROUP BY and HAVING.
7. What is NULL?
8. Define a single-row function.
9. What is an aggregate function?
10. What is NVL?

5-mark questions

1. Explain SQL language categories with examples.
2. Explain SQL constraints.
3. Differentiate DELETE and TRUNCATE.
4. Explain SELECT, WHERE, BETWEEN and ORDER BY.
5. Explain GROUP BY and HAVING with examples.
6. Explain character functions with examples.
7. Explain numeric functions with examples.
8. Explain aggregate functions.
9. Explain Oracle date functions and special date formats.

10-mark questions

1. Explain DDL and DML commands with syntax and examples.
2. Explain SQL query structure and major SELECT features.
3. Explain SQL functions and classify them with suitable examples.
4. Explain character, numeric, conversion, aggregate and date functions with examples.

Module 3 – Functional Dependencies and Normalization

3.1 Redundancy and Its Problems

The normalization slides begin with redundancy and show that repeated information can create:

- **redundant storage,**
- **update anomalies,**
- **insertion anomalies,**
- **deletion anomalies.**

For the supplied Hourly_Emps example:

Hourly_Emps(*ssn, name, lot, rating, hrly_wages, hrs_worked*)

with the notation SNLRWH, the slides state:

$ssn \rightarrow SNLRWH, \quad rating \rightarrow hrly_wages.$

The second dependency causes repeated wage information for the same rating.

3.2 Update, Insertion and Deletion Anomalies

Update anomaly: changing a wage associated with a rating may require changing several tuples.

Insertion anomaly: if a new employee has a rating but the wage for that rating is not known, insertion can become problematic.

Deletion anomaly: deleting all employees with a particular rating can accidentally delete the only stored information about that rating's wage.

Remember

Normalization is fundamentally about reducing problematic redundancy while preserving the information and dependencies needed by the database.

3.3 Functional Dependency

A functional dependency (FD) exists when one set of attributes determines another.

Notation:

$$X \rightarrow Y.$$

If two records have the same value for X , their values for Y must be the same.

Example from the slides:

$$\text{employee number} \rightarrow \text{email address}.$$

Trivial and non-trivial FD

Trivial FD: $X \rightarrow Y$ where $Y \subseteq X$.

Example:

$$\{A, B\} \rightarrow A.$$

Non-trivial FD: $X \rightarrow Y$ where $Y \not\subseteq X$.

Example:

$$A \rightarrow B.$$

3.4 Reasoning About Functional Dependencies

The supplied slides give:

$$ssn \rightarrow did, \quad did \rightarrow lot \quad \Rightarrow \quad ssn \rightarrow lot.$$

An FD f is implied by a set of FDs F if f holds whenever all FDs in F hold.

The **closure of F** , written F^+ , is the set of all FDs implied by F .

3.5 Armstrong's Axioms

The supplied slides identify three core inference rules.

1. **Reflexivity:** if $X \supseteq Y$, then $X \rightarrow Y$.
2. **Augmentation:** if $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
3. **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

The slides state that these are sound and complete inference rules for FDs.

Additional rules:

- **Union:** $X \rightarrow Y$ and $X \rightarrow Z$ imply $X \rightarrow YZ$.
- **Decomposition:** $X \rightarrow YZ$ implies $X \rightarrow Y$ and $X \rightarrow Z$.

3.6 Attribute Closure

The closure of an attribute set X , written X^+ with respect to F , is the set of all attributes functionally determined by X .

Method

1. Start with $X^+ = X$.
2. Find any FD whose left side is contained in the current closure.
3. Add its right-side attributes.
4. Repeat until no new attributes can be added.

Source example

For

$$R(A, B, C, D, F)$$

with

$$F = \{A \rightarrow B, BC \rightarrow D, F \rightarrow C, D \rightarrow A\},$$

the slides work through:

$$A^+ = \{A, B\},$$

$$BC^+ = \{A, B, C, D\},$$

and

$$AF^+ = \{A, B, C, F\}.$$

The key point is not to stop after one dependency; continue applying FDs until closure stops growing.

3.7 Candidate Keys

A **candidate key** is a minimal set of attributes whose closure contains all attributes of the relation.

To test a candidate:

$$X^+ = R$$

means X is a superkey. If no proper subset of X is also a superkey, X is a candidate key.

3.8 Prime and Non-Prime Attributes

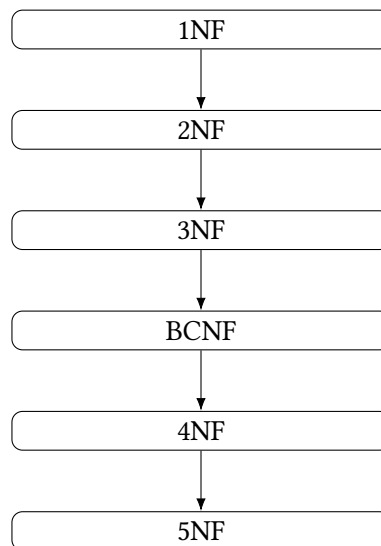
Prime attributes: attributes that occur in at least one candidate key.

Non-prime attributes: attributes that do not occur in any candidate key.

These definitions are repeatedly used in the 2NF and 3NF conditions.

3.9 Normal Forms

The supplied slides present normalization as schema refinement. Higher normal forms represent increasingly restrictive structures.



The slides state that each higher level is a subset of the lower level.

3.10 First Normal Form (1NF)

A table is in 1NF if all fields contain only scalar values rather than lists of values.

A non-1NF repeating group can be converted by:

1. placing repeating-group items in a new table,
2. designating a primary key for the new table,
3. duplicating the original table's primary key in the new table so the relationship can be represented.

Do Not Confuse

The central exam phrase is: **1NF removes repeating groups / non-atomic values and requires scalar values in fields.**

3.11 Second Normal Form (2NF)

The supplied slides give two requirements:

1. The database is in 1NF.
2. Every non-key attribute is fully functionally dependent on the entire primary/candidate key.

Thus, there must be no partial dependency of a non-prime attribute on a proper subset of a candidate key.

Partial dependency

A partial dependency exists if the left side of an FD is a proper subset of a candidate key and the right side is a non-prime attribute.

If a relation has at least one such partial dependency, it is not in 2NF.

Source example

$$R(ABCDEF)$$

$$F = \{C \rightarrow F, E \rightarrow A, EC \rightarrow D, A \rightarrow B\}$$

The supplied slides identify EC as a candidate key and E, C as prime attributes; A, B, D, F are non-prime.

Because $C \rightarrow F$ has a proper subset of the key on the left and a non-prime attribute on the right, the relation is not in 2NF.

3.12 Third Normal Form (3NF)

The supplied slides define 3NF as:

1. the relation is already in 2NF,
2. every non-key attribute depends only on the key and not on another non-key attribute,
3. i.e. transitive dependencies must be removed.

Formal condition from the supplied slides: for every FD $X \rightarrow A$ in F^+ , at least one of the following holds:

1. $X \rightarrow A$ is trivial, or
2. X contains a key for the relation, or
3. A is part of some key (prime attribute).

Transitive dependency

If:

$$X \rightarrow Y, \quad Y \rightarrow Z,$$

then Z may depend transitively on X .

Source example:

$$\{Title, PubID\} \rightarrow PageCount, \quad PageCount \rightarrow Price.$$

Then:

$$\{Title, PubID\} \rightarrow Price$$

transitively, so the relation is not in 3NF.

3.13 BCNF

Boyce–Codd Normal Form is presented as stricter than 3NF.

Basic condition:

For every non-trivial FD $X \rightarrow Y$, X must be a superkey.

The supplied slides summarize this as: the left side/determinant of every FD should be a candidate key/superkey.

Source example

For:

$$R(A, G, N, R)$$

as represented in the source using attributes such as R, N, A, G and FDs

$$R \rightarrow N, \quad R \rightarrow A, \quad A \rightarrow G, \quad A \rightarrow R,$$

the slides identify candidate keys and use the determinant condition to assess BCNF.

3.14 3NF vs BCNF

Feature	3NF	BCNF
Relative strength	Less strict	Stricter
FD condition	LHS is superkey OR RHS is prime	LHS must be superkey
Lossless decomposition	Possible	Possible
Dependency preservation	Slides state a 3NF decomposition can preserve dependencies	Slides state BCNF decomposition may fail to preserve dependencies

Remember

The fastest exam distinction:

3NF: superkey on LHS OR prime attribute on RHS

BCNF: superkey on LHS

3.15 Decomposition

Decomposition breaks a large relation into smaller relations while preserving information and reducing redundancy.

The supplied slides emphasize that decompositions should be used when needed.

Example:

$SNLRWH$

has:

$S \rightarrow SNLRWH, \quad R \rightarrow W.$

The second dependency creates repeated wage information. The slides decompose into:

$SNLRH$ and RW .

3.16 Types of Decomposition

Type	Meaning in the supplied slides
Lossless / lossless-join	No data is lost; the original relation can be reconstructed by joining decomposed relations.
Lossy	Some data may be lost or misinterpreted; the original table cannot be fully recovered by joins.
Dependency-preserving	Functional dependencies of the original relation remain enforceable using the decomposed relations.

3.17 Lossless-Join Decomposition

For decomposition of R into X and Y , the supplied slides give a closure-based condition. The decomposition is lossless with respect to F when the closure contains one of the conditions corresponding to the common attributes determining one side.

They also state:

$$Att(R_1) \cup Att(R_2) = Att(R)$$

and the intersection should not be empty:

$$Att(R_1) \cap Att(R_2) \neq \emptyset.$$

The source further notes that decompositions used to deal with redundancy should be lossless.

3.18 Dependency Preservation

A decomposition is dependency-preserving if the dependencies in the decomposed relations together imply the dependencies of the original relation:

$$(F_X \cup F_Y)^+ = F^+.$$

The supplied slides state:

- 3NF decomposition preserves dependencies.
- BCNF decomposition does not always preserve dependencies.

3.19 Fourth Normal Form (4NF)

A relation is in 4NF if it is in BCNF and has no problematic non-trivial multivalued dependencies.

A multivalued dependency is written:

$$X \twoheadrightarrow Y.$$

The supplied example describes an employee with multiple mobile numbers and multiple email IDs that are independent of each other:

$$Employee_ID \twoheadrightarrow Mobile_Number$$

$$Employee_ID \twoheadrightarrow Email_ID.$$

The resulting redundancy can be removed through decomposition.

3.20 Decomposition for 4NF

The supplied slides state that if $X \twoheadrightarrow Y$ violates 4NF, decompose using the same general technique:

$$XY$$

as one relation and the remaining attributes with X as the other relation.

3.21 Fifth Normal Form (5NF)

5NF is also called **Projection-Join Normal Form (PJNF)**.

The supplied slides define 5NF as:

- already being in 4NF,
- having no join dependencies that cause data redundancy.

A join dependency indicates that a relation can be losslessly reconstructed from multiple projections/relations.

3.22 Minimal / Canonical Cover

A minimal cover (canonical cover) is an equivalent set of FDs that is:

- minimal in size,
- free of extraneous attributes,
- represented with singleton right-hand sides.

Steps

1. **Convert to singleton RHS:** split $A \rightarrow BC$ into $A \rightarrow B$ and $A \rightarrow C$.
2. **Remove redundant FDs:** remove an FD if its removal does not change the closure.
3. **Remove extraneous attributes:** remove an attribute from an FD if doing so does not change the closure.

Source example

Given:

$$A \rightarrow BC, \quad B \rightarrow C, \quad A \rightarrow D, \quad D \rightarrow E,$$

split:

$$A \rightarrow B, \quad A \rightarrow C, \quad B \rightarrow C, \quad A \rightarrow D, \quad D \rightarrow E.$$

Since $A \rightarrow B$ and $B \rightarrow C$ already imply $A \rightarrow C$, remove $A \rightarrow C$.

Final minimal cover:

$$A \rightarrow B, \quad B \rightarrow C, \quad A \rightarrow D, \quad D \rightarrow E.$$

The supplied slide then gives a canonical-cover form:

$$A \rightarrow BD, \quad B \rightarrow C, \quad D \rightarrow E.$$

Source Note

For normalization problems, do not memorize only definitions. Practice attribute closure, candidate-key discovery, prime/non-prime classification, 2NF/3NF/BCNF checking, decomposition, lossless-join reasoning, dependency preservation, and minimal-cover construction.

Module 3 – Worked Exam Frameworks

3.23 How to Check a Relation for Normal Form

1. Write the relation schema R and FD set F .
2. Compute candidate keys using attribute closure.
3. Mark prime and non-prime attributes.
4. Check 1NF: all attributes/fields are atomic.
5. Check 2NF: look for partial dependency of a non-prime attribute on a proper subset of a candidate key.
6. Check 3NF: for every relevant FD, verify the superkey/prime-RHS condition.
7. Check BCNF: every determinant must be a superkey.
8. If required, examine MVDs for 4NF.
9. If required, examine join dependencies for 5NF.

Exam Focus

High-yield workflow:

FDs $\rightarrow$ Closures $\rightarrow$ Candidate Keys $\rightarrow$ Prime Attributes $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF

3.24 Quick Normal-Form Comparison

Normal form	Core condition to remember
1NF	Atomic/scalar values; no repeating groups.
2NF	1NF + no partial dependency of non-prime attributes on a proper subset of a candidate key.
3NF	2NF + for each FD, LHS is a superkey or RHS is prime.
BCNF	For each non-trivial FD, LHS is a superkey.
4NF	BCNF + no non-trivial multivalued dependency violating the form.
5NF	4NF + no problematic join dependencies causing redundancy.

3.25 Important Practice Problems from the Slides

1. Find candidate keys, prime attributes, and non-prime attributes for:

$$R(ABCDEF), \quad F = \{AB \rightarrow C, C \rightarrow DE, E \rightarrow F, F \rightarrow A\}.$$

2. Find the highest normal form for:

$$R(ABCDEF), \quad F = \{AB \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F, F \rightarrow E\}.$$

3. Check 3NF:

$$R(ABCD), \quad F = \{AB \rightarrow C, C \rightarrow D\}.$$

4. Check 3NF:

$$R(ABCD), \quad F = \{AB \rightarrow CD, D \rightarrow A\}.$$

5. Find a suitable decomposition for:

$$R(ABCDE), \quad F = \{A \rightarrow C, B \rightarrow DE, D \rightarrow C\}.$$

6. Find a decomposition for:

$$R(ABCDE), \quad F = \{A \rightarrow B, B \rightarrow E, C \rightarrow D\}.$$

7. Find a minimal cover for:

$$A \rightarrow BC, \quad B \rightarrow C, \quad A \rightarrow D, \quad D \rightarrow E.$$

4. 10-Mark Answer Templates

Question A: Explain DBMS and its advantages

[10 marks]

Answer Skeleton

Write in this order:

1. Define data, database, and DBMS.
2. Explain the database approach.
3. Explain disadvantages of traditional file systems.
4. List DBMS advantages: independence, efficient access, reduced development time, integrity, security, administration, concurrency, recovery.
5. Draw a simple User/Application ↔ DBMS ↔ Database diagram.
6. Conclude with why centralized database management is useful.

Question B: Explain ER modelling

[10 marks]

Answer Skeleton

1. Define ER modelling.
2. Explain entity and entity set.
3. Explain relationship and relationship set.
4. Explain cardinality.
5. Explain attributes and their types.
6. Explain strong and weak entities.
7. Draw a small Chen ERD and label the symbols.
8. Mention specialization/generalization and aggregation.

Question C: Explain SQL commands

[10 marks]

Answer Skeleton

1. Define SQL.
2. Classify DDL, DML, DCL and TCL.
3. Give CREATE TABLE syntax.
4. Explain constraints.
5. Give ALTER and DROP examples.
6. Explain INSERT, UPDATE, DELETE and TRUNCATE.
7. Explain SELECT and WHERE with one query.

Question D: Explain SQL functions

[10 marks]

Answer Skeleton

1. Define SQL function.
2. Classify single-row and multiple-row functions.
3. Explain character functions with examples.
4. Explain numeric functions with examples.
5. Explain conversion/general functions.
6. Explain aggregate functions.
7. Explain date functions and TO_CHAR date formatting.

Question E: Explain normalization and normal forms**[10 marks]****Answer Skeleton**

1. Define redundancy and anomalies.
2. Define functional dependency.
3. Explain Armstrong's axioms.
4. Explain attribute closure and candidate keys.
5. Explain 1NF.
6. Explain 2NF and partial dependency.
7. Explain 3NF and transitive dependency.
8. Explain BCNF.
9. Add 4NF/5NF if asked or if marks permit.
10. Draw the normal-form progression.

5. High-Yield Comparison Tables

5.1 File System vs DBMS

Feature	Traditional file approach	DBMS approach
Data organization	Separate files per application	Shared related data
Redundancy	Can be high	Reduced through centralized management/normalization
Consistency	Duplicate updates can diverge	Integrity mechanisms available
Concurrent access	Can cause conflicts	DBMS provides controlled concurrent access
Security	More limited	Access-control mechanisms available
Recovery	More application-dependent	DBMS supports recovery from crashes

5.2 DDL vs DML

Feature	DDL	DML
Main concern	Structure/schema	Data
Examples	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE, SELECT
Typical effect	Creates/modifies database objects	Reads/inserts/changes/removes rows

5.3 WHERE vs HAVING

Feature	WHERE	HAVING
Filters	Rows	Groups
Timing in supplied explanation	Before grouping	After grouping
Typical use	Row condition	Aggregate/group condition

5.4 Single-Row vs Aggregate Functions

Feature	Single-row	Multiple-row / aggregate
Input	One row at a time	Multiple rows / group
Output	One result per row	One result per group/result set
Examples	LOWER, UPPER, ROUND, SUBSTR	AVG, MIN, MAX, COUNT, SUM

5.5 2NF vs 3NF vs BCNF

2NF	Remove partial dependency of non-prime attributes on a proper subset of a candidate key.
3NF	For every FD, LHS is a superkey OR RHS is prime.
BCNF	For every non-trivial FD, LHS is a superkey.

6. Last-Minute Revision Page

DBMS CORE

Database = organized collection of data. DBMS = programs that manipulate the database and provide an interface to users/applications.

ABSTRACTION

Physical = storage. Logical = data + relationships. View = user/application view.

ERD

Rectangle = entity. Diamond = relationship. Oval = attribute. Underline = key. Double rectangle = weak entity.

CARDINALITY

1:1, 1:M, M:1, M:N. Minimum 0 = optional; minimum 1 = mandatory.

RELATIONAL MAPPING

Entity $\rightarrow$ table. 1:M $\rightarrow$ FK on many side. M:N $\rightarrow$ new table with both FKs.

SQL GROUPING

GROUP BY creates groups. WHERE filters rows. HAVING filters groups.

SQL SETS

UNION = combine; INTERSECT = common; MINUS/EXCEPT = first minus second.

NULL

Use IS NULL / IS NOT NULL. Arithmetic involving NULL produces NULL in the supplied material.

FUNCTIONS

Single-row: one result per row. Aggregate: one result per group/result set.

CHARACTER

LOWER, INITCAP, UPPER, SUBSTR, ASCII, LENGTH, LTRIM, RTRIM, TRIM, LPAD, RPAD, VSIZE, INSTR.

NUMERIC

ABS, POWER, ROUND, SQRT, EXP, EXTRACT, GREATEST, LEAST, MOD, TRUNC, FLOOR, CEIL.

AGGREGATE

AVG, MIN, COUNT, COUNT(), MAX, SUM.*

DATE

SYSDATE, ADD_MONTHS, MONTHS_BETWEEN, NEXT_DAY, LAST_DAY, ROUND, TRUNC.

FD

$X \rightarrow Y$: X determines Y.

ARMSTRONG

Reflexivity, Augmentation, Transitivity.

CLOSURE

Start with X; repeatedly apply FDs until no new attributes appear.

CANDIDATE KEY

Minimal attribute set whose closure is the whole relation.

PRIME ATTRIBUTE

Appears in at least one candidate key.

1NF

Atomic/scalar values; no repeating groups.

2NF

1NF + no partial dependency of non-prime attributes.

3NF

LHS superkey OR RHS prime.

BCNF

Every non-trivial FD has a superkey on the LHS.

4NF

BCNF + no problematic non-trivial MVD.

5NF

4NF + no problematic join dependency.

DECOMPOSITION

Aim for lossless join; preserve dependencies when required.

MINIMAL COVER

Singleton RHS $\rightarrow$ remove redundant FDs $\rightarrow$ remove extraneous attributes.

7. Rapid-Fire Question Bank

Module 1

1. Define DBMS.
2. Explain data hierarchy.
3. What is data independence?
4. Explain physical, logical and view levels.
5. Define schema and instance.
6. Define entity, relationship and attribute.
7. Explain cardinality.
8. Explain strong and weak entities.
9. Explain specialization and generalization.
10. Explain ER-to-relational mapping.

Module 2

1. What is SQL?
2. Explain DDL/DML/DCL/TCL.
3. Write CREATE TABLE syntax.
4. Explain constraints.
5. Write ALTER TABLE examples.
6. Differentiate DELETE and TRUNCATE.
7. Explain SELECT, WHERE, ORDER BY.
8. Explain GROUP BY and HAVING.
9. Explain set operations.
10. Explain NULL.
11. List character functions.
12. Explain numeric functions.
13. Explain aggregate functions.
14. Explain conversion and date functions.

Module 3

1. What is redundancy?
2. Explain insertion, deletion and update anomalies.
3. Define functional dependency.
4. Explain trivial and non-trivial FDs.
5. Explain Armstrong's axioms.
6. Find attribute closure.
7. Find candidate keys.
8. Define prime and non-prime attributes.
9. Explain 1NF, 2NF and 3NF.
10. Explain BCNF.
11. Differentiate 3NF and BCNF.
12. Explain lossless decomposition.
13. Explain dependency preservation.
14. Explain 4NF and MVD.
15. Explain 5NF and join dependency.
16. Find a minimal/canonical cover.

8. Exam Writing Checklist

- Start every long answer with a clear definition.
- Use headings instead of writing one large paragraph.

- Draw ER symbols when explaining ER modelling.
- For SQL questions, write syntax first and then one example.
- For normalization problems, always show X^+ steps rather than jumping to the key.
- Explicitly identify candidate keys and prime attributes before checking 2NF/3NF.
- For 3NF, write the two-part condition: LHS superkey **or** RHS prime.
- For BCNF, write the stricter single condition: LHS must be a superkey.
- When decomposing, mention lossless join and dependency preservation if relevant.

Source Scope

These notes were generated from the supplied lecture files:

- s#1 Intro.pptx – Database Concepts
- S#3 ERD.pptx – E–R Modelling
- s#3 SQL_Intro.pptx – SQL
- S#4 SQL_Functions.pptx – SQL Functions
- s#8 Normalization.pptx – Database Normalization

Source Note

Terminology, examples, emphasis, and SQL/Oracle syntax are based on the supplied slides. Where a supplied slide contains an apparent wording inconsistency, it is explicitly flagged rather than silently corrected. Image-only slide content was incorporated where readable from the supplied slide visuals.

Study principle: understand the mechanism first, then memorize the exam wording.